

Final Project

Ava J. Lindgren

CS-330: Comp Graphic and Visualization

Professor Thomas Gerlach

June 21, 2026

Final Project

In this project, I developed a 3D scene that represents a simplified urban environment, specifically Chicago's Millennium Park, using basic geometric primitives. I chose objects such as a lamp post, buildings, traffic cones, and a ground plane because they are commonly found in real-world city settings and could be effectively modeled using low-polygon shapes. The lamp post and the Bean became the focal points of my scene which helped me in guiding my layout decisions and object placement. I built complex objects by combining simpler primitives, like stacking cylinders for the lamp post and using a sphere to represent the lamp bulb. These development choices allowed me to meet the project requirements while keeping the scene efficient and visually organized. I also focused on ensuring that textures and lighting enhanced realism without overcomplicating the geometry. For all of my textures, I sourced all of them through ambientcg.com, and as for the light, I struggled endlessly with it while using the Codio Virtual Machine, so I am actually unsure if my lighting displays correctly despite working on my code and improving it as much as I can for it to render correctly.

To navigate the 3D scene, the user is able to move through the environment using keyboard and mouse input. The WASD keys control forward, backward, and lateral movement, then the Q and E keys allow vertical movement along the Y-axis. The mouse is used to control the orientation of the camera, allowing the user to look around the scene freely. Additionally, the scroll wheel can be used to adjust movement speed, giving the user more control over how quickly they explore the environment. The scene also includes a toggle between perspective and orthographic projection, which changes how the world is visually represented without altering object positions. All of these controls allows the user to explore the 3D environment from multiple viewpoints.

The virtual camera in the project is managed through a dedicated camera system that handles the position, direction and the projection. Camera movement is processed through input callbacks that update the camera's position based on keyboard events, while mouse movement adjusts the viewing direction using yaw and pitch calculations. This allows for smooth and continuous navigation through my 3D environment. The camera is also configured with adjustable movement speed and zoom level, which improves usability and allows the user to fine-tune their viewing experience. These controls are processed each frame to ensure responsive interaction with the environment.

To keep the code both organized and reusable, I used several custom functions within the `SceneManager` and `ViewManager` classes. In `SetTransformations`, I applied scaling, rotation, and translation to any object needed in the scene, making it reusable for every mesh without rewriting transformation logic. I also used `SetShaderTexture` and `SetShaderColor` to allow me to switch between textured and non-textured objects easily while keeping rendering code consistent. The `PrepareScene` function was used to load all textures, materials, and lighting settings in one place, which keeps initialization clean and modular. All of these functions work together and improve readability and also make it easier to adjust or expand the scene without modifying core rendering logic.

Although I struggled a lot with this project, I believe that it demonstrates fairly well how modular design and structured input handling can be used to build an interactive 3D environment. I combined reusable functions, proper camera controls, as well as carefully choosing the primitives, and was able to construct a cohesive scene following the requirements of the project. The organization of the code also makes it easier to maintain and extend in future projects, especially if additional objects, lighting effects, or camera behaviors are added.

References

AmbientCG - free textures, HDRIs and models. (n.d.). <http://ambientcg.com/>

LearnOpenGL - Basic Lighting. (n.d.). <https://learnopengl.com/Lighting/Basic-Lighting>

LearnOpenGL - Textures. (n.d.). <https://learnopengl.com/Getting-started/Textures>